

Roteiro — Aula 1: back-end, Ruby e o primeiro Rails

Deck: `slides/build/aula-01-fundamentos-de-back-end.pptx` (62 slides) **Apostila da turma:** `01-fundamentos.md` · **Checkpoint:** `aula-01`

Antes de começar

Na semana anterior

- [] Mandar `00-preparacao.md` no grupo e cobrar resposta de quem travou.
- [] Levantar quem usa Windows → WSL2 instalado **antes**.
- [] Cobrar a conta Azure for Students (a verificação demora dias e é da Aula 4).

No dia, antes de a turma chegar

- [] Deck aberto, modo apresentador (as notas de cada slide estão preenchidas).
 - [] Dois terminais grandes: um para o `irb`, outro para o `rails`.
 - [] Fonte do terminal em pelo menos 18pt.
 - [] Insomnia aberto.
 - [] `curl -i https://api.seemxxiii.tech/api/v1/status` testado — é a sua demo do slide 26.
 - [] `git checkout aula-01` num diretório separado, pronto para socorrer quem travar.
-

Cronograma

Relógio	Slides	Bloco	Min
00:00	1–2	Abertura e objetivos	5
00:05	3–10	Ferramentas e setup	23
00:28	11–15	O que é back-end	12
00:40	16–19	Rede: servidor, IP, porta, DNS, URL	13
00:53	20–27	HTTP e JSON	22
01:15	28–29	API REST	10
01:25	—	Intervalo	10
01:35	30–45	Ruby para quem sabe Python	42
02:17	46	Prática 1 — <code>irb</code>	10
02:27	47–58	Rails e a primeira rota	25
02:52	59	Prática 2 — <code>GET /api/v1/status</code>	30
03:22	60–62	Git, recapitulação, fim	5

Isso dá 3h27. Ou você corta, ou aceita passar. Onde cortar, em ordem:

1. Slide 6 (`O TERMINAL`) e 60 (`GIT`) — os dois marcados `# CORTÁVEL` . **–7 min**
2. Slides 40–43 (argumentos nomeados, `Struct` , exceções) — explique quando aparecerem na Aula 3. **–12 min**
3. Slide 18 (`DNS`) — volta na Aula 4 de qualquer jeito. **–3 min**

Cortando os três, fecha em 03:05.

Bloco a bloco

00:00 — Abertura (slides 1–2)

Você se apresenta e diz de onde veio a capacitação: **é a continuação da do Fiuza**. Lá foi o que o usuário vê; aqui é o outro lado.

Nos objetivos, a frase que prende: *"no fim do encontro 4, cada um de vocês vai ter uma URL `https://` própria, funcionando, que qualquer um do mundo consegue chamar."*

00:05 — Ferramentas e setup (3–10)

Passe rápido pelos slides e **pare no 10**. Todo mundo roda os quatro comandos ao mesmo tempo:

```
curl https://mise.run | sh
mise use --global ruby@3.4.9
gem install rails -v 8.1.3
docker run --rm hello-world
```

Peça para todos mostrarem a saída de `ruby -v` antes de seguir.

Ponto de não-retorno: se passou de 00:35 e ainda tem gente instalando, **pare**. Pareie quem travou com quem terminou e siga. Instalação é problema de casa, não de aula.

00:28 — O que é back-end (11–15)

O slide 12 é a ponte com a capacitação anterior: mostre a coluna do front e diga "isso vocês já viram". O slide 13 tem o ponto que importa: **tudo que roda no navegador o usuário consegue ler e alterar** — por isso a validação que vale é a do back.

O restaurante (14) funciona bem. Volte nele quando falar de API.

00:40 — Rede (16–19)

Aqui muita gente descobre coisa que usava sem saber.

- **16:** servidor não é máquina especial, é programa esperando numa porta.
- **17:** a tabela de portas. Diga que 22, 80 e 443 voltam na Aula 4, quando forem abrir e fechar firewall na mão.
- **19:** desmonte a URL na tela. Pergunte: "por que `localhost:3000` tem dois pontos e `google.com` não?" Deixe alguém responder.

00:53 — HTTP (20–27)

Slide 21 é o coração do bloco. Uma requisição HTTP é texto puro. Se der, faça ao vivo:

```
curl -v https://api.seemxxiii.tech/api/v1/status
```

O `-v` mostra as linhas com `>` (o que foi) e `<` (o que voltou). É a mesma coisa do slide, acontecendo de verdade.

No **23** (`Content-Type`), avise: "esquecer esse cabeçalho é o erro nº 1 de vocês na Aula 3. Vem 400 e a mensagem não ajuda em nada."

No **25** (status codes), a pergunta: "qual a diferença entre 401 e 403?" — 401 é "quem é você?", 403 é "sei quem você é, e você não pode". Diga que cai em entrevista.

No **fim do bloco**, marque a frase: **HTTP não tem memória**. Escreva no quadro se tiver. É o problema inteiro da Aula 3.

01:15 — API REST (28–29)

Curto. O que fica: o verbo é a ação, o endereço é a coisa. `/criarPalestra` está errado; `POST /lectures` está certo.

O 29 explica por que o SEEM é uma API: um back-end servindo o app e o painel.

01:35 — Ruby (30–45)

O bloco mais longo. **Não leia os slides — rode no `irb` ao vivo.**

```
3.class          # Integer
"oi".class       # String
:oi.class        # Symbol
nil.class        # NilClass

3.times { puts "oi" }

usuario = { nome: "Ana", role: :admin }
usuario[:nome]

[1, 2, 3].map { |n| n * n }
[1, 2, 3].select { |n| n.odd? }
```

Pontos onde vale parar:

- **33** — as três diferenças: `end`, `if` no fim da linha, retorno implícito.
- **35** — símbolo não existe em Python. A regra: conteúdo que o usuário lê é string; nome de coisa no código é símbolo.
- **38** — `map` / `select` são iguais aos de Python. A diferença é que aqui são o caminho normal.
- **39** — o `?` e o `!`. Diga que o Rails inteiro depende dessa convenção.
- **40–41** — argumentos nomeados e `Struct`. Fale a frase: "o `user:` sozinho não é erro de digitação", porque eles vão ver isso em todo service da Aula 3.

02:17 — Prática 1 (46)

Dez minutos no `irb`. Circule pela sala. Quem terminar rápido, mande fazer o bônus (`map` para elevar ao quadrado).

02:27 — Rails (47–58)

- **49** é a ideia central: você não configura o óbvio, você segue o combinado.
- **51** (Django × Rails) — pergunte quem já usou Django ou Flask e ancore neles.
- **54** (Zeitwerk) — a frase: "não é estilo, é mecanismo. Errou o caminho, a classe não existe."
- **58** — mostre a rota e o controller lado a lado, e aponte a convenção ligando os dois.

02:52 — Prática 2 (59)

A entrega da aula. **Ninguém sai sem os 200 na tela.**

```
rails new automic_auth_api --api -d postgresql \
  --skip-action-mailbox --skip-action-text --skip-active-storage \
  --skip-jbuilder --skip-action-cable
cd automic_auth_api
docker compose up -d
bin/rails db:prepare
bin/rails server
```

Depois a rota, e o teste no Insomnia.

Quem travar: `git checkout aula-01`.

03:22 — Fecho (60–62)

Recapitule em cinco frases (slide 61) e feche com o que vem: *"na próxima, a API ganha memória."*

Perguntas que vão aparecer

Pergunta	Resposta curta
"Por que Ruby e não Python?"	Porque o <code>seem-backend</code> é Ruby, e porque Rails entrega um sistema inteiro sem você montar peça por peça. E a linguagem é o de menos: o que vocês vão aprender aqui vale em qualquer uma.
"Rails não morreu?"	GitHub, Shopify e Basecamp rodam nele hoje. O que morreu foi o hype, não a ferramenta.
"Preciso decorar os status codes?"	Não. Precisa saber a faixa: 2xx deu certo, 4xx você errou, 5xx eu errei. O resto se consulta.
"Por que não usar o navegador para testar a API?"	O navegador só sabe fazer GET. Nossas rotas são POST e DELETE.
"Posso usar o Ruby que já está instalado no meu Linux?"	Pode dar certo e pode dar errado. É por isso que existe o mise: a versão exata, por projeto.
"O <code>--api</code> tira alguma coisa que vou precisar?"	Tira view, sessão de navegador e assets. Se um dia precisar, dá para trazer de volta — foi o que fizemos com os cookies.

Dever de casa

1. Terminar a prática, se não deu tempo.
2. Ler `ruby-para-pythonistas.md` inteiro.
3. Bônus do slide 59: fazer a rota devolver `RUBY_VERSION` e `Rails.version`.